

# TÀI LIỆU KIẾN TRÚC TỔNG QUÁT: BẢO MẬT DỮ LIỆU ĐA TẦNG (DATA SECURITY ARCHITECTURE)

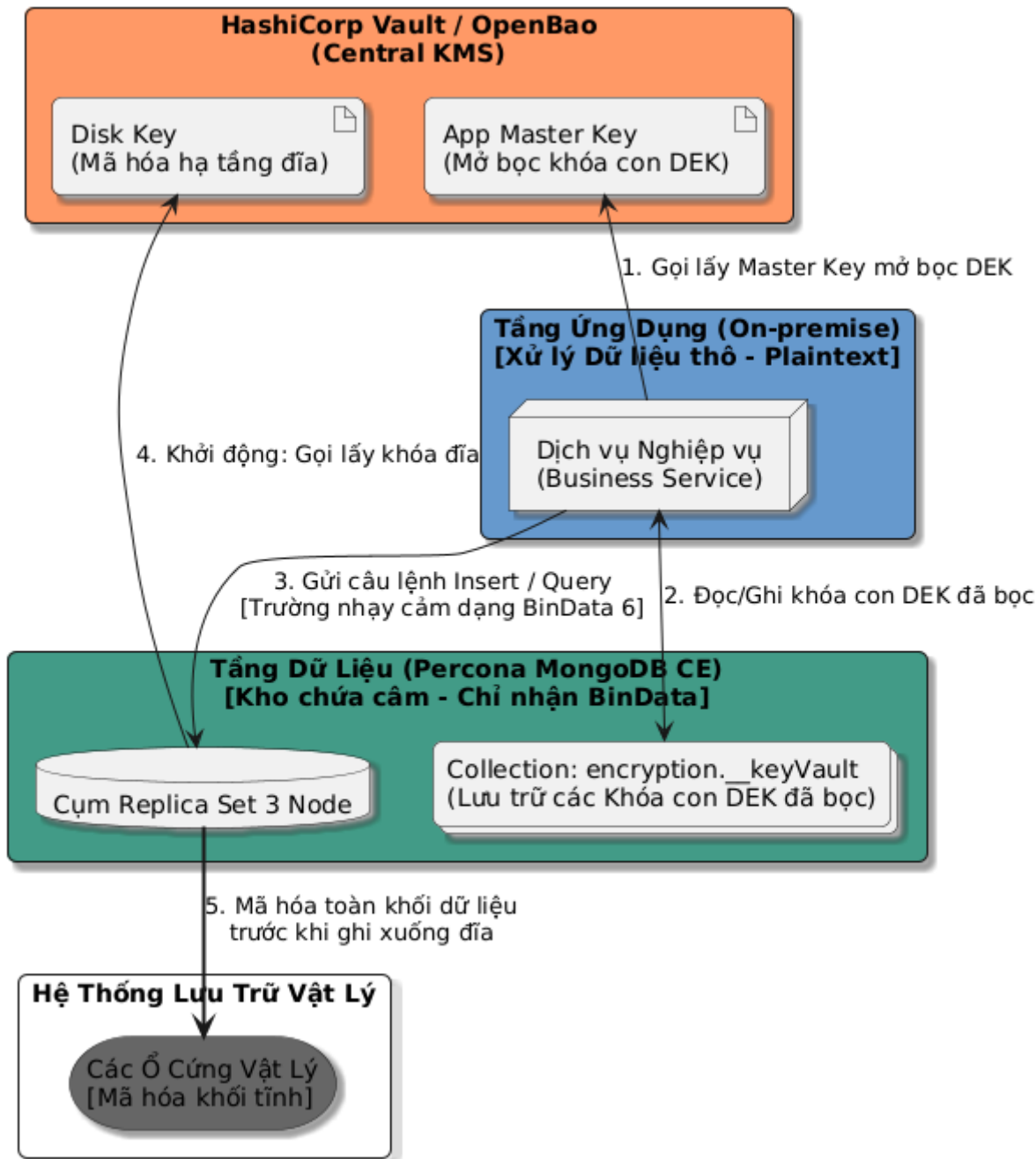
Tài liệu này áp dụng với công nghệ Hashicop Vault và Percona MongoDB CE

## I. TỔNG QUAN KIẾN TRÚC (ARCHITECTURE OVERVIEW)

Hệ thống áp dụng mô hình bảo mật "**Zero-Trust Data**" bằng cách kết hợp hai giải pháp mã hóa độc lập để bảo vệ dữ liệu ở mọi trạng thái:

- Mã hóa cấp trường phía khách (CSFLE):** Thực hiện tại tầng **Dịch vụ Nghiệp vụ (Business Service)**. Dữ liệu nhạy cảm được mã hóa trước khi truyền qua mạng và ghi vào DB. Chống rò rỉ dữ liệu ngay cả khi Quản trị viên DB (DBA) hoặc Hacker chiếm được quyền root của hệ quản trị cơ sở dữ liệu.
- Mã hóa dữ liệu tĩnh (Data-at-Rest Encryption):** Thực hiện tại tầng Lưu trữ (Storage Engine) của Percona MongoDB. Toàn bộ file dữ liệu vật lý lưu trên ổ cứng sẽ được mã hóa, chống rò rỉ dữ liệu khi bị đánh cắp ổ đĩa vật lý hoặc sao chép trộm file database.

Tất cả các khóa mẹ (Master Keys) đều khiến hai tính năng trên được quản lý tập trung và nghiêm ngặt tại cụm **HashiCorp Vault**.



```
@startuml
skinparam backgroundColor white
skinparam roundcorner 10
skinparam shadowing true

' Định nghĩa các dải màu phân vùng hệ thống
!define VAULT_COLOR #F96
!define APP_COLOR #69C
!define DB_COLOR #439A86
!define DISK_COLOR #666

rectangle "HashiCorp Vault / OpenBao\n(Central KMS)" as Vault_Zone #F96 {
    artifact "App Master Key\n(Mở bọc khóa con DEK)" as AppMasterKey
    artifact "Disk Key\n(Mã hóa hạ tầng đĩa)" as DBDiskKey
}
```

```

rectangle "Tầng Ứng Dụng (On-premise)\n[Xử lý Dữ liệu thô - Plaintext]" as
App_Zone #69C {
    node "Dịch vụ Nghiệp vụ\n(Business Service)" as BusinessService
}

rectangle "Tầng Dữ Liệu (Percona MongoDB CE)\n[Kho chứa câm - Chỉ nhận
BinData]" as DB_Zone #439A86 {
    database "Cụm Replica Set 3 Node" as MongoCluster
    collections "Collection: encryption.__keyVault\n(Lưu trữ các Khóa con
DEK đã bọc)" as KeyVaultColl
}

rectangle "Hệ Thống Lưu Trữ Vật Lý" as Disk_Zone {
    storage "Các Ổ Cứng Vật Lý\n[Mã hóa khối tĩnh]" as HardDisks #666
}

' Thể hiện các luồng tương tác tổng quát
BusinessService -up-> AppMasterKey : "1. Gọi lấy Master Key mở bọc DEK"
BusinessService <--> KeyVaultColl : "2. Đọc/Ghi khóa con DEK đã bọc"

BusinessService --> MongoCluster : "3. Gửi câu lệnh Insert / Query\n[Trường
nhập cảm dạng BinData 6]"
MongoCluster -up-> DBDiskKey : "4. Khởi động: Gọi lấy khóa đĩa"
MongoCluster ==> HardDisks : "5. Mã hóa toàn khối dữ liệu\ntrước khi ghi
xuống đĩa"

@enduml

```

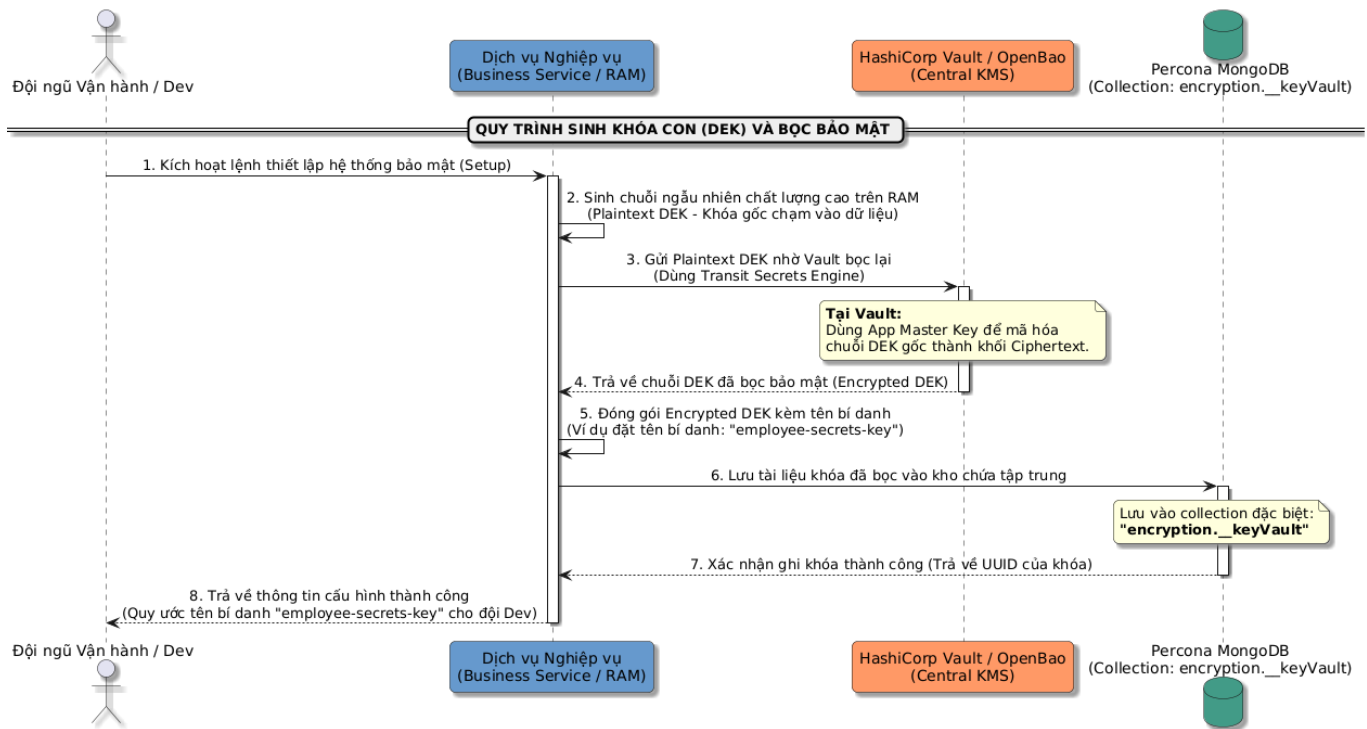
## II. THIẾT KẾ CHI TIẾT

### 1. Mã hóa cấp trường phía khách (CSFLE):

#### 1.1. Khởi tạo và sinh khóa con (DEK Generation)

Trước khi ứng dụng có thể mã hóa dữ liệu, nó tự sinh ra con ngẫu nhiên (Plaintext DEK), gửi lên Vault để bọc bảo mật bằng Master Key, sau đó lưu vào bộ lưu trữ tập trung của MongoDB dưới dạng đã bọc (Encrypted

DEK).



```

@startuml
skinparam backgroundColor white
skinparam roundcorner 10
skinparam shadowing true
skinparam SequenceMessageAlign center
  
```

```

actor "Đội ngũ Vận hành / Dev" as Admin
participant "Dịch vụ Nghiệp vụ\n(Business Service / RAM)" as App #69C
participant "HashiCorp Vault / OpenBao\n(Central KMS)" as Vault #F96
database "Percona MongoDB\n(Collection: encryption.__keyVault)" as DB #439A86
  
```

```

== QUY TRÌNH SINH KHÓA CON (DEK) VÀ BỘC BẢO MẬT ==
  
```

```

Admin -> App : 1. Kích hoạt lệnh thiết lập hệ thống bảo mật (Setup)
activate App
  
```

```

App -> App : 2. Sinh chuỗi ngẫu nhiên chất lượng cao trên RAM\n(Plaintext DEK - Khóa gốc chạm vào dữ liệu)
  
```

```

App -> Vault : 3. Gửi Plaintext DEK nhờ Vault bọc lại\n(Dùng Transit Secrets Engine)
activate Vault
note over Vault
  **Tại Vault:**
  Dùng App Master Key để mã hóa
  chuỗi DEK gốc thành khối Ciphertext.
end note
  
```

```

Vault --> App : 4. Trả về chuỗi DEK đã bọc bảo mật (Encrypted DEK)
deactivate Vault
  
```

App -> App : 5. Đóng gói Encrypted DEK kèm tên bí danh\n(Ví dụ đặt tên bí danh: "employee-secrets-key")

App -> DB : 6. Lưu tài liệu khóa đã bọc vào kho chứa tập trung  
activate DB  
note over DB

Lưu vào collection đặc biệt:

\*\*\*"encryption.\_\_keyVault"\*\*\*

end note

DB --> App : 7. Xác nhận ghi khóa thành công (Trả về UUID của khóa)  
deactivate DB

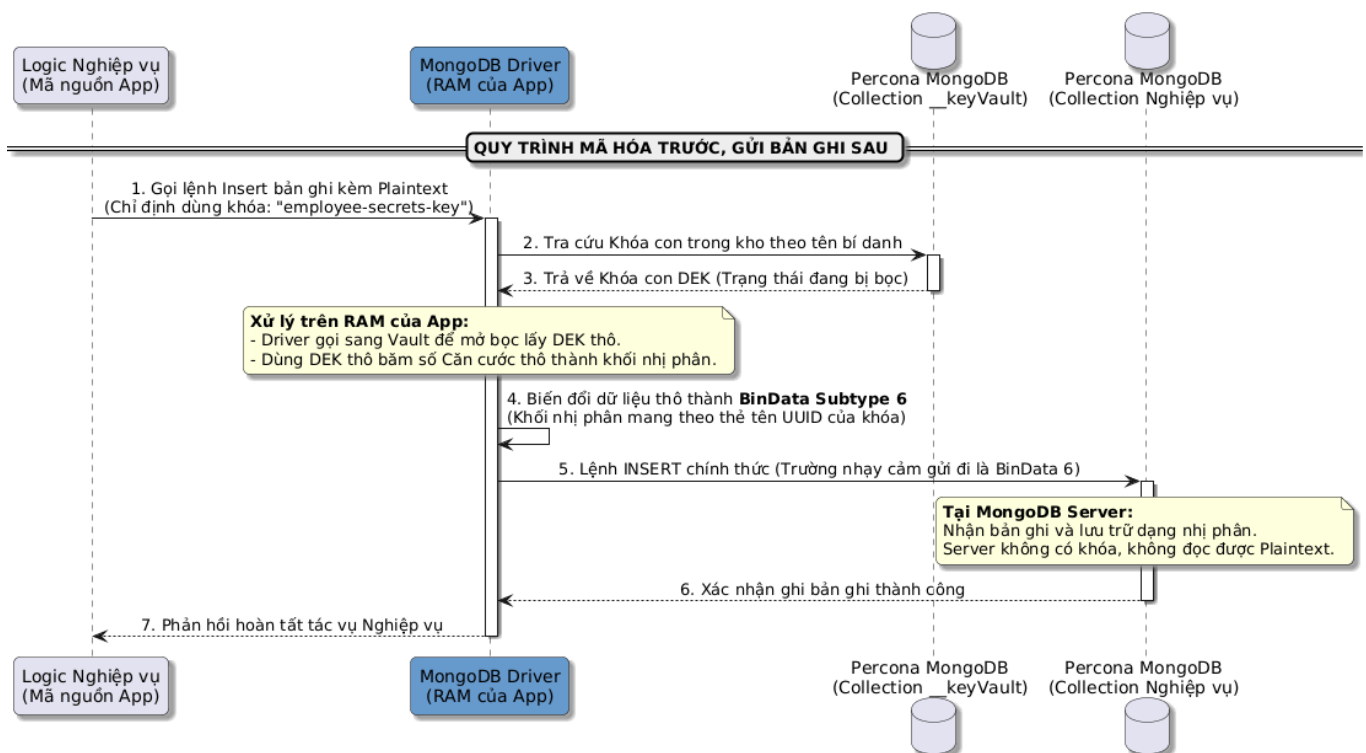
App --> Admin : 8. Trả về thông tin cấu hình thành công\n(Quy ước tên bí danh "employee-secrets-key" cho đội Dev)

deactivate App

@startuml

## 1.2. Mã hóa và ghi mới dữ liệu (CSFLE Insert Data)

Khi thực hiện ghi bản ghi mới, Dịch vụ Nghiệp vụ tìm khóa con bằng tên bí danh, mã hóa trường thông tin nhạy cảm thành chuỗi nhị phân BinData Subtype 6 trên RAM rồi mới đẩy câu lệnh qua mạng tới MongoDB.



@startuml

skinparam backgroundColor white

skinparam roundcorner 10

skinparam shadowing true

skinparam SequenceMessageAlign center

participant "Logic Nghiệp vụ\n(Mã nguồn App)" as Logic

participant "MongoDB Driver\n(RAM của App)" as Driver #69C

```

database "Percona MongoDB\n(Collection __keyVault)" as DBKey
database "Percona MongoDB\n(Collection Nghiệp vụ)" as DBData

== QUY TRÌNH MÃ HÓA TRƯỚC, GỬI BẢN GHI SAU ==

Logic -> Driver : 1. Gọi lệnh Insert bản ghi kèm Plaintext\n(Chỉ định dùng
khóa: "employee-secrets-key")
activate Driver

Driver -> DBKey : 2. Tra cứu Khóa con trong kho theo tên bí danh
activate DBKey
DBKey --> Driver : 3. Trả về Khóa con DEK (Trạng thái đang bị bọc)
deactivate DBKey

note over Driver
  **Xử lý trên RAM của App:**
  - Driver gọi sang Vault để mở bọc lấy DEK thô.
  - Dùng DEK thô bấm số Căn cước thô thành khối nhị phân.
end note
Driver -> Driver : 4. Biến đổi dữ liệu thô thành **BinData Subtype
6**\n(Khối nhị phân mang theo thẻ tên UUID của khóa)

Driver -> DBData : 5. Lệnh INSERT chính thức (Trường nhảy cảm gửi đi là
BinData 6)
activate DBData
note over DBData
  **Tại MongoDB Server:**
  Nhận bản ghi và lưu trữ dạng nhị phân.
  Server không có khóa, không đọc được Plaintext.
end note
DBData --> Driver : 6. Xác nhận ghi bản ghi thành công
deactivate DBData

Driver --> Logic : 7. Phản hồi hoàn tất tác vụ Nghiệp vụ
deactivate Driver
@enduml

```

#### Mã nguồn Ví dụ Java (CSFLE Auto Insert):

Dưới đây là mã nguồn Java hoàn chỉnh để cấu hình **MongoClient** với tính năng tự động mã hóa dữ liệu khi ghi mới. Hệ thống sử dụng **KMS Provider** dạng **local** và nạp Master Key được quản lý, lấy trực tiếp từ bộ nhớ đệm (RAM) sau khi đọc từ **HashiCorp Vault**.

```

package com.company.security;

import com.mongodb.AutoEncryptionSettings;
import com.mongodb.ClientEncryptionSettings;
import com.mongodb.ConnectionString;
import com.mongodb.MongoClientSettings;
import com.mongodb.client.MongoClient;
import com.mongodb.client.MongoClients;

```

```
import com.mongodb.client.MongoCollection;
import com.mongodb.client.MongoDatabase;
import com.mongodb.client.model.vault.DataKeyOptions;
import com.mongodb.client.vault.ClientEncryption;
import com.mongodb.client.vault.ClientEncryptions;
import org.bson.*;
import org.bson.codecs.configuration.CodecRegistry;
import org.bson.codecs.configuration.CodecRegistries;
import org.bson.codecs.UuidCodec;
import java.util.*;

public class MongoCsfleInsertExample {

    private static final String CONNECTION_STRING =
"mongodb://admin:secret@localhost:27017/?replicaSet=rs0";
    private static final String KEY_VAULT_DB = "encryption";
    private static final String KEY_VAULT_COLL = "__keyVault";
    private static final String KEY_VAULT_NAMESPACE = KEY_VAULT_DB + "." +
KEY_VAULT_COLL;

    private static final String DB_NAME = "companyDb";
    private static final String COLLECTION_NAME = "employees";

    public static void main(String[] args) {
        // 1. Lấy Master Key 96 bytes từ HashiCorp Vault (qua SDK/REST API)
        byte[] masterKey = fetchMasterKeyFromVault();

        // 2. Cấu hình KMS Provider (Sử dụng 'local' với Master Key từ
Vault)
        Map<String, Map<String, Object>> kmsProviders = new HashMap<>();
        Map<String, Object> localProvider = new HashMap<>();
        localProvider.put("key", masterKey);
        kmsProviders.put("local", localProvider);

        // 3. Khởi tạo/Tìm khóa con Data Encryption Key (DEK)
        UUID dekUuid = getOrCreateDataEncryptionKey(kmsProviders);
        System.out.println("DEK UUID: " + dekUuid);

        // 4. Tạo JSON Schema định nghĩa trường tự động mã hóa
        Map<String, BsonDocument> schemaMap =
createAutoEncryptionSchema(dekUuid);

        // 5. Cấu hình UuidRepresentation STANDARD để đảm bảo định dạng
UUID
        CodecRegistry codecRegistry = CodecRegistries.fromRegistries(
            CodecRegistries.fromCodecs(new
UuidCodec(UuidRepresentation.STANDARD)),
            MongoClientSettings.getDefaultCodecRegistry()
        );

        // 6. Thiết lập AutoEncryptionSettings
        AutoEncryptionSettings autoEncryptionSettings =
AutoEncryptionSettings.builder()
            .keyVaultNamespace(KEY_VAULT_NAMESPACE)
    }
```

```

        .kmsProviders(kmsProviders)
        .schemaMap(schemaMap)
        .build();

    MongoClientSettings clientSettings = MongoClientSettings.builder()
        .applyConnectionString(new
ConnectionString(CONNECTION_STRING))
        .autoEncryptionSettings(autoEncryptionSettings)
        .codecRegistry(codecRegistry)
        .build();

    // 7. Khởi tạo MongoClient có khả năng mã hóa tự động
    try (MongoClient mongoClient = MongoClients.create(clientSettings))
    {
        MongoDB database = mongoClient.getDatabase(DB_NAME);
        MongoCollection<Document> collection =
database.getCollection(COLLECTION_NAME);

        // Xóa dữ liệu cũ của collection nghiệp vụ để chạy thử
        collection.drop();

        // 8. Đưa tài liệu dạng thô (Plaintext) vào.
        // Driver sẽ dựa vào schemaMap để tự động mã hóa các trường đã
        chỉ định thành BinData Subtype 6 trước khi truyền đi.
        Document employee = new Document()
            .append("name", "Nguyễn Văn A")
            .append("personalID", "0123456789") // Sẽ được tự động
        mã hóa Deterministic (Hỗ trợ tìm kiếm)
            .append("phoneNumber", "0901234567"); // Sẽ được tự
        động mã hóa Random (Tăng cường bảo mật)

        collection.insertOne(employee);
        System.out.println("Ghi dữ liệu mới thành công! Dữ liệu đã tự
        động mã hóa phía Client.");
    }
}

/**
 * Giả lập việc kết nối và đọc Master Key 96 bytes từ HashiCorp Vault.
 */
private static byte[] fetchMasterKeyFromVault() {
    // Master Key của MongoDB CSFLE (Local KMS) bắt buộc phải đủ 96
    bytes.
    // Bạn có thể dùng Vault SDK (com.bettercloud.vault-java-driver)
    hoặc HTTP REST API
    // để đọc khóa tĩnh mã hóa từ Vault Transit/KV Engine và gán vào
    đây.

    byte[] masterKey = new byte[96];
    new Random().nextBytes(masterKey); // Trả về mock key phục vụ demo
    return masterKey;
}

/**
 * Lấy hoặc Sinh mới Data Encryption Key (DEK) có bí danh trong

```



```

KeyVault
    */
    private static UUID getOrCreateDataEncryptionKey(Map<String,
Map<String, Object>> kmsProviders) {
        MongoClientSettings clientSettings = MongoClientSettings.builder()
            .applyConnectionString(new
ConnectionString(CONNECTION_STRING))
            .build();

        try (MongoClient keyVaultClient =
MongoClients.create(clientSettings)) {
            ClientEncryptionSettings clientEncryptionSettings =
ClientEncryptionSettings.builder()
                .keyVaultMongoClientSettings(clientSettings)
                .keyVaultNamespace(KEY_VAULT_NAMESPACE)
                .kmsProviders(kmsProviders)
                .build();

            try (ClientEncryption clientEncryption =
ClientEncryptions.create(clientEncryptionSettings)) {
                MongoCollection<Document> keyVaultColl = keyVaultClient
                    .getDatabase(KEY_VAULT_DB)
                    .getCollection(KEY_VAULT_COLL);

                Document query = new Document("keyAltNames", "employee-
secrets-key");
                Document existingKey = keyVaultColl.find(query).first();

                if (existingKey != null) {
                    return (UUID) existingKey.get("_id");
                } else {
                    BsonBinary keyId = clientEncryption.createDataKey(
                        "local",
                        new
DataKeyOptions().keyAltNames(Collections.singletonList("employee-secrets-
key")))
                        );
                    return keyId.asUuid();
                }
            }
        }
    }

    /**
     * Định nghĩa BSON Schema Map để chỉ rõ trường và thuật toán mã hóa
     */
    private static Map<String, BsonDocument>
createAutoEncryptionSchema(UUID dekUuid) {
        BsonBinary keyId = new BsonBinary(dekUuid);
        BsonArray keyIds = new BsonArray(Collections.singletonList(keyId));

        // 1. Căn cước (personalID): Mã hóa Deterministic hỗ trợ khớp truy
vấn chính xác
        BsonDocument personalIdSchema = new BsonDocument()

```

```

        .append("encrypt", new BsonDocument()
            .append("keyId", keyIds)
            .append("bsonType", new BsonString("string"))
            .append("algorithm", new
BsonString("AEAD_AES_256_CBC_HMAC_SHA_512-Deterministic"))
        );

        // 2. Số điện thoại (phoneNumber): Mã hóa Random (Mỗi lần mã hóa ra
        kết quả khác nhau) để bảo mật tối đa
        BsonDocument phoneNumberSchema = new BsonDocument()
            .append("encrypt", new BsonDocument()
                .append("keyId", keyIds)
                .append("bsonType", new BsonString("string"))
                .append("algorithm", new
BsonString("AEAD_AES_256_CBC_HMAC_SHA_512-Random"))
            );

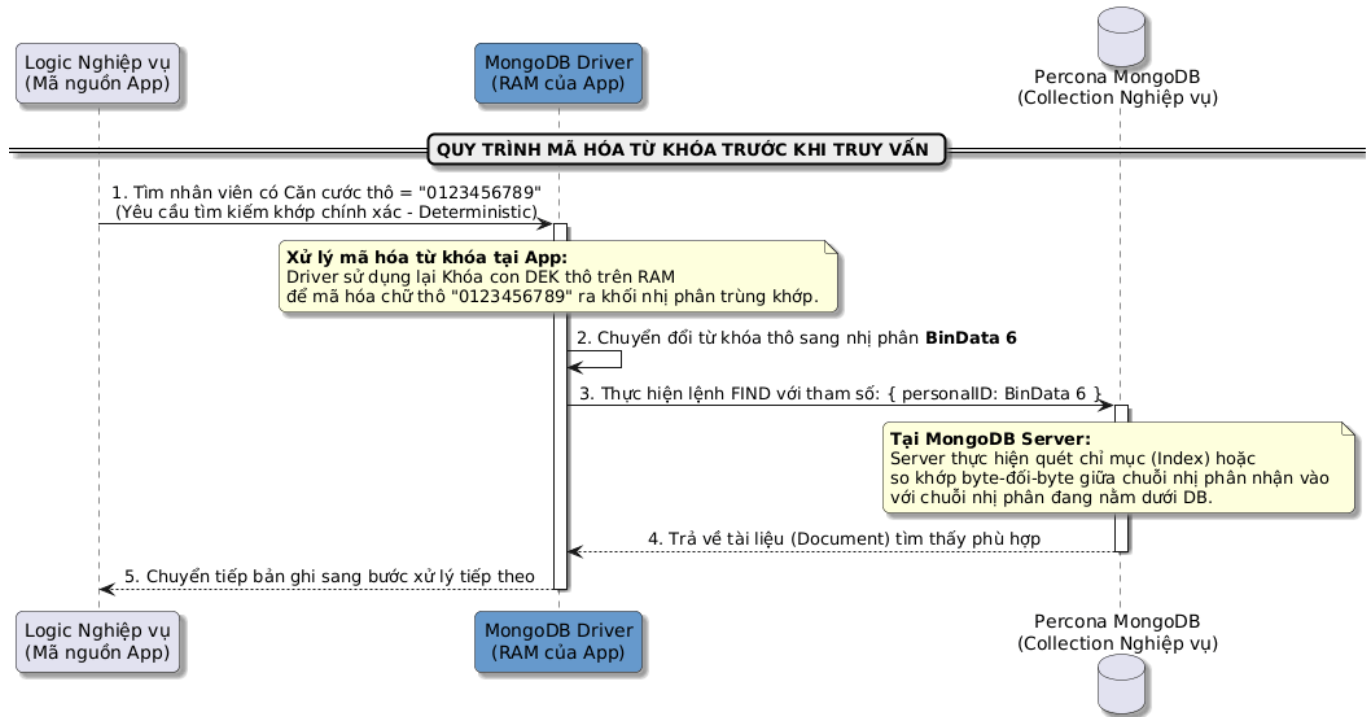
        BsonDocument schema = new BsonDocument()
            .append("bsonType", new BsonString("object"))
            .append("properties", new BsonDocument()
                .append("personalID", personalIdSchema)
                .append("phoneNumber", phoneNumberSchema)
            );

        Map<String, BsonDocument> schemaMap = new HashMap<>();
        schemaMap.put(DB_NAME + "." + COLLECTION_NAME, schema);
        return schemaMap;
    }
}

```

### 1.3. Mã hóa giá trị tìm kiếm (CSFLE Query / Find Data)

Do dữ liệu dưới database là chuỗi nhị phân, ứng dụng không thể tìm kiếm theo dạng chữ thô. Nó buộc phải mã hóa từ khóa tìm kiếm thành dạng nhị phân tương ứng (sử dụng thuật toán Deterministic) trước khi gửi truy vấn sang MongoDB Server để thực hiện so khớp byte-đối-byte.



```
@startuml
skinparam backgroundColor white
skinparam roundcorner 10
skinparam shadowing true
skinparam SequenceMessageAlign center
```

```
participant "Logic Nghiệp vụ\n(Mã nguồn App)" as Logic
participant "MongoDB Driver\n(RAM của App)" as Driver #69C
database "Percona MongoDB\n(Collection Nghiệp vụ)" as DBData
```

```
== QUY TRÌNH MÃ HÓA TỪ KHÓA TRƯỚC KHI TRUY VẤN ==
```

```
Logic -> Driver : 1. Tìm nhân viên có Căn cước thô = "0123456789"\n(Yêu cầu tìm kiếm khớp chính xác - Deterministic)
activate Driver
```

```
note over Driver
```

```
  **Xử lý mã hóa từ khóa tại App:**
```

```
  Driver sử dụng lại Khóa con DEK thô trên RAM
```

```
  để mã hóa chữ thô "0123456789" ra khối nhị phân trùng khớp.
```

```
end note
```

```
Driver -> Driver : 2. Chuyển đổi từ khóa thô sang nhị phân **BinData 6**
```

```
Driver -> DBData : 3. Thực hiện lệnh FIND với tham số: { personalID:  
BinData 6 }
```

```
activate DBData
```

```
note over DBData
```

```
  **Tại MongoDB Server:**
```

```
  Server thực hiện quét chỉ mục (Index) hoặc
```

```
  so khớp byte-đối-byte giữa chuỗi nhị phân nhận vào
```

```
  với chuỗi nhị phân đang nằm dưới DB.
```

```
end note
```

```

DBData --> Driver : 4. Trả về tài liệu (Document) tìm thấy phù hợp
deactivate DBData

Driver --> Logic : 5. Chuyển tiếp bản ghi sang bước xử lý tiếp theo
deactivate Driver
@enduml

```

#### Mã nguồn Ví dụ Java (CSFLE Auto Query & Read):

Dưới đây là mã nguồn Java thể hiện cách tìm kiếm bằng giá trị thô (**Plaintext**). Nhờ thuật toán mã hóa đồng nhất (**Deterministic**) khai báo trong schema, **MongoDB Driver** tự động mã hóa giá trị tìm kiếm sang **BinData** nhị phân trước khi gửi đi, đồng thời tự động giải mã bản ghi trả về cho ứng dụng:

```

package com.company.security;

import com.mongodb.AutoEncryptionSettings;
import com.mongodb.ConnectionString;
import com.mongodb.MongoClientSettings;
import com.mongodb.client.MongoClient;
import com.mongodb.client.MongoClients;
import com.mongodb.client.MongoCollection;
import com.mongodb.client.MongoDatabase;
import com.mongodb.client.model.Filters;
import org.bson.*;
import org.bson.codecs.UuidCodec;
import org.bson.codecs.configuration.CodecRegistry;
import org.bson.codecs.configuration.CodecRegistries;
import java.util.*;

public class MongoCsfleQueryExample {

    private static final String CONNECTION_STRING =
"mongodb://admin:secret@localhost:27017/?replicaSet=rs0";
    private static final String KEY_VAULT_DB = "encryption";
    private static final String KEY_VAULT_COLL = "__keyVault";
    private static final String KEY_VAULT_NAMESPACE = KEY_VAULT_DB + "." +
KEY_VAULT_COLL;

    private static final String DB_NAME = "companyDb";
    private static final String COLLECTION_NAME = "employees";

    public static void main(String[] args) {
        // 1. Khởi tạo Master Key từ Vault (phải tương đồng với Master Key
lúc sinh DEK)
        byte[] masterKey = fetchMasterKeyFromVault();
        Map<String, Map<String, Object>> kmsProviders = new HashMap<>();
        Map<String, Object> localProvider = new HashMap<>();
        localProvider.put("key", masterKey);
        kmsProviders.put("local", localProvider);

        // 2. Tra cứu tìm UUID của DEK đã có trong DB

```

```

        UUID dekUuid = getExistingDataEncryptionKey();
        System.out.println("Sử dụng DEK UUID hiện tại: " + dekUuid);

        // 3. Khởi tạo Schema Map hỗ trợ giải mã và mã hóa tự động
        Map<String, BsonDocument> schemaMap =
            createAutoEncryptionSchema(dekUuid);

        CodecRegistry codecRegistry = CodecRegistries.fromRegistries(
            CodecRegistries.fromCodecs(new
            UuidCodec(UuidRepresentation.STANDARD)),
            MongoClientSettings.getDefaultCodecRegistry()
        );

        AutoEncryptionSettings autoEncryptionSettings =
            AutoEncryptionSettings.builder()
                .keyVaultNamespace(KEY_VAULT_NAMESPACE)
                .kmsProviders(kmsProviders)
                .schemaMap(schemaMap)
                .build();

        MongoClientSettings clientSettings = MongoClientSettings.builder()
            .applyConnectionString(new
            ConnectionString(CONNECTION_STRING))
            .autoEncryptionSettings(autoEncryptionSettings)
            .codecRegistry(codecRegistry)
            .build();

        try (MongoClient mongoClient = MongoClient.create(clientSettings))
        {
            MongoDBDatabase database = mongoClient.getDatabase(DB_NAME);
            MongoCollection<Document> collection =
                database.getCollection(COLLECTION_NAME);

            //
            =====
            // 1.3. THỰC HIỆN TRUY VẤN (QUERY/FIND DATA)
            //
            =====
            // Tìm kiếm nhân viên bằng Căn cước thô: "0123456789".
            // Do trường 'personalID' có cấu hình mã hóa 'Deterministic',
            Driver sẽ tự động
            // chuyển đổi giá trị thô sang BinData nhị phân trùng khớp hoàn
            toàn để so sánh trên chỉ mục (Index).
            String searchId = "0123456789";
            System.out.println("Đang truy vấn nhân viên có personalID
            (Plaintext): " + searchId);

            Document employee = collection.find(Filters.eq("personalID",
            searchId)).first();

            if (employee != null) {
                //
                =====
                // 1.4. GIẢI MÃ TỰ ĐỘNG (DECRYPT / READ DATA)

```

```

//
=====
// Khi nhận kết quả dạng nhị phân BinData Subtype 6, Driver
tự động:
// - Trích xuất UUID của DEK nằm trong trường BinData.
// - Mở bọc DEK và giải mã chuỗi nhị phân về Plaintext gốc
trên RAM của App.
System.out.println("\n=== KẾT QUẢ TÌM THẤY & GIẢI MÃ TỰ
ĐỘNG ===");
        System.out.println("Tên nhân viên          : " +
employee.getString("name"));
        System.out.println("Căn cước (personalID)    : " +
employee.getString("personalID"));
        System.out.println("Số điện thoại (phone)   : " +
employee.getString("phoneNumber"));
    } else {
        System.out.println("Không tìm thấy nhân viên với Căn cước
chỉ định!");
    }
}
}

private static byte[] fetchMasterKeyFromVault() {
    // Thực tế: Lấy Master Key tính từ Vault Transit/KV để bọc và mở
bọc
    byte[] masterKey = new byte[96];
    // Đảm bảo byte array này khớp hoàn toàn với key đã sinh ra DEK lúc
Insert
    return masterKey;
}

private static UUID getExistingDataEncryptionKey() {
    MongoClientSettings clientSettings = MongoClientSettings.builder()
        .applyConnectionString(new
ConnectionString(CONNECTION_STRING))
        .build();
    try (MongoClient keyVaultClient =
MongoClients.create(clientSettings)) {
        MongoCollection<Document> keyVaultColl = keyVaultClient
            .getDatabase(KEY_VAULT_DB)
            .getCollection(KEY_VAULT_COLL);

        Document query = new Document("keyAltNames", "employee-secrets-
key");
        Document keyDoc = keyVaultColl.find(query).first();
        if (keyDoc == null) {
            throw new IllegalStateException("Không tìm thấy Data
Encryption Key bí danh employee-secrets-key trong __keyVault!");
        }
        return (UUID) keyDoc.get("_id");
    }
}

private static Map<String, BsonDocument>

```

```

createAutoEncryptionSchema(UUID dekUuid) {
    BsonBinary keyId = new BsonBinary(dekUuid);
    BsonArray keyIds = new BsonArray(Collections.singletonList(keyId));

    BsonDocument personalIdSchema = new BsonDocument()
        .append("encrypt", new BsonDocument()
            .append("keyId", keyIds)
            .append("bsonType", new BsonString("string"))
            .append("algorithm", new
BsonString("AEAD_AES_256_CBC_HMAC_SHA_512-Deterministic")))
        );

    BsonDocument phoneNumberSchema = new BsonDocument()
        .append("encrypt", new BsonDocument()
            .append("keyId", keyIds)
            .append("bsonType", new BsonString("string"))
            .append("algorithm", new
BsonString("AEAD_AES_256_CBC_HMAC_SHA_512-Random")))
        );

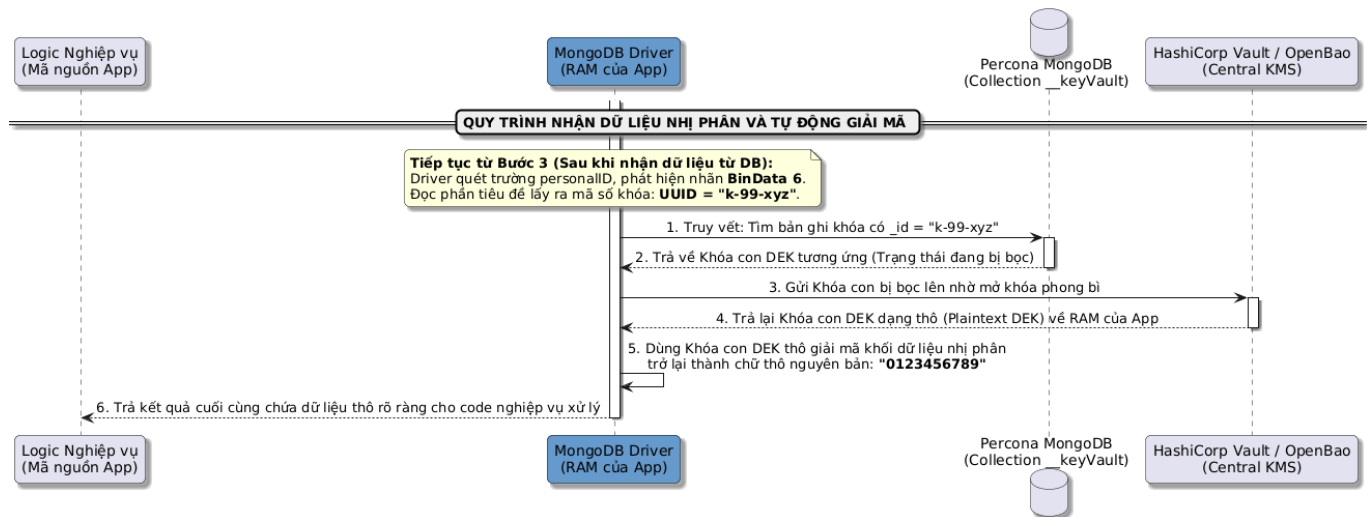
    BsonDocument schema = new BsonDocument()
        .append("bsonType", new BsonString("object"))
        .append("properties", new BsonDocument()
            .append("personalID", personalIdSchema)
            .append("phoneNumber", phoneNumberSchema)
        );

    Map<String, BsonDocument> schemaMap = new HashMap<>();
    schemaMap.put(DB_NAME + "." + COLLECTION_NAME, schema);
    return schemaMap;
}
}

```

#### 1.4. Giải mã và trả về dữ liệu (CSFLE Read Data)

Khi nhận khối nhị phân đổ về từ MongoDB, Driver chạy trên RAM của ứng dụng tự động bóc tách tiêu đề dữ liệu để lấy mã UUID (Key ID), tìm đúng bản ghi khóa con trong \_\_keyVault, gửi sang Vault mở bọc và dịch ngược dữ liệu về chữ thô ban đầu.



```
@startuml
skinparam backgroundColor white
skinparam roundcorner 10
skinparam shadowing true
skinparam SequenceMessageAlign center

participant "Logic Nghiệp vụ\n(Mã nguồn App)" as Logic
participant "MongoDB Driver\n(RAM của App)" as Driver #69C
database "Percona MongoDB\n(Collection __keyVault)" as DBKey
participant "HashiCorp Vault / OpenBao\n(Central KMS)" as Vault

== QUY TRÌNH NHẬN DỮ LIỆU NHỊ PHÂN VÀ TỰ ĐỘNG GIẢI MÃ ==

activate Driver
note over Driver
  **Tiếp tục từ Bước 3 (Sau khi nhận dữ liệu từ DB):**
  Driver quét trường personalID, phát hiện nhãn **BinData 6**.
  Đọc phần tiêu đề lấy ra mã số khóa: **UUID = "k-99-xyz"**.
end note

Driver -> DBKey : 1. Truy vết: Tìm bản ghi khóa có _id = "k-99-xyz"
activate DBKey
DBKey --> Driver : 2. Trả về Khóa con DEK tương ứng (Trạng thái đang bị bọc)
deactivate DBKey

Driver -> Vault : 3. Gửi Khóa con bị bọc lên nhờ mở khóa phong bì
activate Vault
Vault --> Driver : 4. Trả lại Khóa con DEK dạng thô (Plaintext DEK) về RAM của App
deactivate Vault

Driver -> Driver : 5. Dùng Khóa con DEK thô giải mã khối dữ liệu nhị phân\ntrở lại thành chữ thô nguyên bản: **"0123456789"**

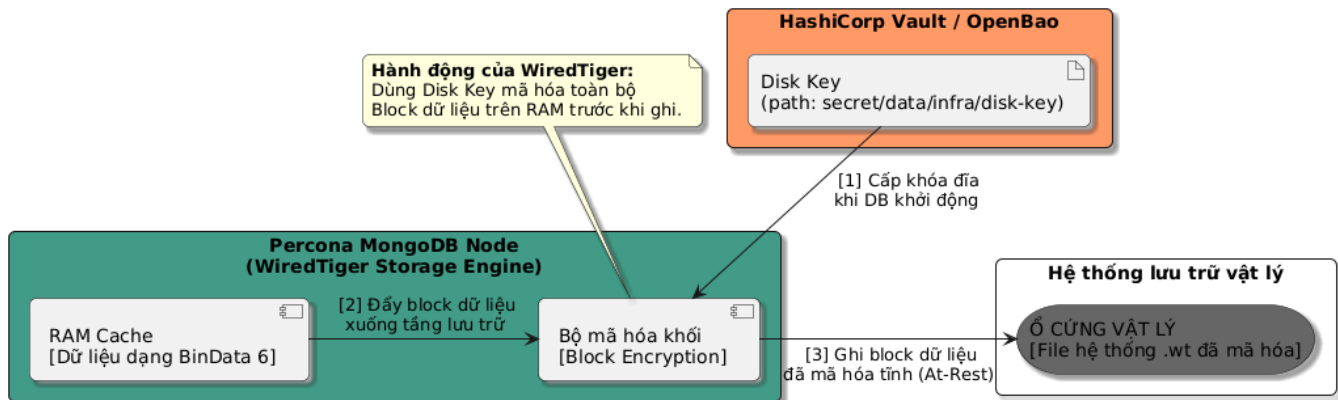
Driver --> Logic : 6. Trả kết quả cuối cùng chứa dữ liệu thô rõ ràng cho code nghiệp vụ xử lý
```



```
deactivate Driver
@enduml
```

## 2. Data-at-Rest Encryption (Mã hóa dữ liệu tĩnh)

Tầng lưu trữ WiredTiger của Percona MongoDB kết nối trực tiếp đến Vault để lấy Disk Key khi khởi động cụm. Quá trình đồng bộ hóa giữa các node Secondary vẫn giữ nguyên trạng thái nhị phân, và hành động mã hóa đĩa diễn ra ngay trước khi các block dữ liệu được đổ xuống ổ cứng vật lý.



```
@startuml
skinparam backgroundColor white
skinparam roundcorner 10
skinparam shadowing true
skinparam orientation left to right

' Định nghĩa màu sắc các thành phần
!define VAULT_COLOR #F96
!define DB_COLOR #439A86
!define DISK_COLOR #666

' 1. Kho khóa trung tâm
rectangle "HashiCorp Vault / OpenBao" as Vault #F96 {
    artifact "Disk Key\n(path: secret/data/infra/disk-key)" as DiskKey
}

' 2. Tầng xử lý cơ sở dữ liệu (RAM/CPU Server)
rectangle "Percona MongoDB Node\n(WiredTiger Storage Engine)" as MongoDBServer #439A86 {
    component "RAM Cache\n[Dữ liệu dạng BinData 6]" as Cache
    component "Bộ mã hóa khối\n[Block Encryption]" as Encryptor
}

' 3. Tầng lưu trữ vật lý (Disk)
rectangle "Hệ thống lưu trữ vật lý" as DiskZone #666 {
    storage "Ổ CỨNG VẬT LÝ\n[File hệ thống .wt đã mã hóa]" as PhysicalDisk
}

' =====
```

```
' Cấu trúc luồng dữ liệu di chuyển (Flow)
' =====

' Luồng lấy khóa khi khởi động hệ thống
DiskKey -down-> Encryptor : " [1] Cấp khóa đĩa\nkhi DB khởi động"

' Luồng xử lý ghi dữ liệu xuống đĩa
Cache -right-> Encryptor : " [2] Đẩy block dữ liệu\nxuống tầng lưu trữ"

note top of Encryptor
  **Hành động của WiredTiger:**
  Dùng Disk Key mã hóa toàn bộ
  Block dữ liệu trên RAM trước khi ghi.
end note

Encryptor -right-> PhysicalDisk : " [3] Ghi block dữ liệu\ndã mã hóa tĩnh
(At-Rest)"

@enduml
```